CLAUDE.md

SECTION 01-governance TYPE document STATUS active OWNER Founder UPDATED 2026-07-03

> **Vendored copy** — this is the application repo's (`FlyGACA/flygaca`) Claude guidance, kept here for reference. Paths like `functions/` , `docs/` , and `office/` refer to that repo's layout, not to this documents repo.

This file provides guidance to Claude Code (claude.ai/code) when working with code in this repository.

What this is

Fly GACA is an independent, educational platform and open regulatory library for Saudi civil aviation (GACAR, AIP, charts, study tools, an AI flight instructor). It is **not affiliated with GACA**; every surface reinforces one rule — verify against the latest official GACA publication, and the product helps you find/study regulation, never replaces it. Treat this constraint as load-bearing when touching user-facing copy or assistant behaviour: the assistant cites the exact Part/section and refuses rather than guesses.

The frontend is a **no-framework static PWA** — vanilla JavaScript (ES2022), HTML5, CSS3. There is **no bundler and no build step for the site itself**; pages are plain `.html` files served directly. The only "build" is a stamper that propagates the shared header/ footer (see below).

Commands

```
# End-to-end tests (Playwright; auto-starts a python static server on :4178)
npm test
npx playwright test tests/smoke.spec.js          # one file
npx playwright test -g "bilingual"               # one test by title
npm run test:headed                              # watch it run
npm run test:report                              # open the last HTML report

# Fast unit tests (node --test, zero deps) — Cloud Functions entitlement/webhook
# invariants (entitlements-core / revenuecat-core / stripe-core) + quota date math
npm run test:unit

# Firestore security-rules tests (spins up the emulator; needs Java 21+)
npm run test:rules

# Integrity / guard checks (all run in CI; no browser, no deps)
npm run check:data          # library index counts, quiz/ground-school structure, README
figures
npm run check:i18n          # bilingual coverage — every data-en has a data-ar (and vice
versa)
npm run check:links         # internal hrefs/srcs resolve, firebase.json routes + sw.js
precache + sitemap
npm run check:content       # triage aid: flags OCR/extraction artifacts in the
regulation corpus
npm run check:sources       # is the local GACA corpus in sync with sources.json
manifest?
npm run update:sources      # fetch + apply official-source updates (see RUNBOOK-source-
updates.md)

# Generated data — definitions/abbreviations glossary index
npm run build:defs
# Fast unit tests (node --test, zero deps): Cloud Functions entitlement/webhook
# core, KSA daily-quota date math, and gateway↔captadel guard/limiter parity
npm run test:unit

# Firestore security-rules tests (firebase-tools emulators:exec; needs Java 21+)
npm run test:rules

# Repo guards — each exits non-zero on a violation (these are the CI checks)
npm run check:data          # library index counts, quiz/ground-school structure, README
figures
npm run check:i18n          # every data-en has a matching data-ar (and vice versa)
npm run check:links         # internal links, firebase routes, sw precache, sitemap all
resolve
npm run check:content       # OCR/extraction-defect triage over the GACAR corpus
(reports, exit 0)
npm run check:sources       # drift-check the GACA source manifest (exit 1 if a source
changed)
npm run update:sources      # download new/changed official docs into library/ staging +
refresh indexes

# Generated artifacts (run after editing their inputs)
npm run build:chrome        # stamp nav/footer across all pages from partials/ — see
"Shared chrome"
```

```
npm run check:chrome        # CI guard: exit 1 if any page is out of sync
npm run build:defs          # rebuild assets/data/definitions-index.json from GACAR Part
1

# iOS / Capacitor wrapper (an Android wrapper also exists under android/)
# iOS / Capacitor wrapper (ios/ is primary; android/ scaffold exists)
npm run build:ios           # assemble the web payload into www/
npm run cap:sync            # build:ios + npx cap sync ios
npm run cap:open

# Serve locally (pages fetch JSON, so file:// will NOT work)
python3 -m http.server 8000        # then open http://localhost:8000/flygaca.html
firebase serve                     # also applies the firebase.json rewrites
firebase emulators:start           # needed to exercise Captain Adel locally

# Deploy (default project flygaca-app; prod is flygaca.com)
firebase deploy                                    # hosting + functions + rules
firebase deploy --only hosting,firestore:rules
```

The standalone Captain Adel service now lives in its **own repository**, `FlyGACA/Captain-Adel` (captadel.com). Clone it separately; its commands run from that repo's root:

```
git clone https://github.com/FlyGACA/Captain-Adel.git
cd Captain-Adel
npm start                          # Express server on :8787
npm run eval:dry                   # eval structure check, no API key needed
GEMINI_API_KEY=… npm run eval      # full live eval (English/Gemini path)
npm run eval:allam                 # eval the Arabic/ALLaM path (also :jais, :fanar,
:qwen, :commandr)
npm run eval:parity                # EN/AR answer-parity check across providers
npm run test:unit                  # Captain Adel unit tests (node --test)
```

Architecture

The repo holds **two deployable units plus the static site**:

1. **The static PWA** (repo root) — production hosting is a **Cloudflare Worker** (`wrangler.jsonc` + `worker/index.js`): it serves the top-level `*.html` pages plus the `tools/`, `guides/`, `study/`, `packs/` subpages from an assembled `dist/` (`scripts/build-cloudflare.js`), applies the clean-URL rewrites (`/library` → `/library.html`), the redirects and the security headers/CSP, and **proxies** `/api/chat` → the `chat` Cloud Function and `/api/content` → `protectedContent` (the functions stay on Firebase me-central2). `firebase.json` mirrors the same routing/ignore set and is retained for `firebase serve` (local dev) and as the rollback host; the worker and `firebase.json` must be kept in sync. Almost everything outside the served pages (`functions/`, `library/`, `office/`, `assistant/`, `docs/`, `partials/`, `scripts/`, `tests/`, all `*.md`) is excluded from `dist/` (and the Hosting `ignore` list) and never deployed. See `office/RUNBOOK-cloudflare.md`.

2. `functions/` — **the Fly GACA gateway** (Firebase Cloud Functions, Node 20). `index.js` exports `chat` plus billing/account/notification functions: `createCheckoutSession` + `stripeWebhook` (web billing), `revenuecatWebhook` + `linkRevenueCatIdentity` (iOS IAP → entitlement), `expiryReminders` (medical/flight-review push), `grantSchoolLicence` + `revokeSchoolLicence` (B2B), `protectedContent` (gated assets). The `chat` function is a thin gateway: it owns auth (verifies the Firebase ID token), entitlement checks, the free-tier daily quota (`rag/dailyquota.js`), the abuse rate-limiter (`rag/ratelimit.js`) and input guards (`rag/guards.js`), then **proxies the turn server-to-server** to the standalone Captain Adel service (`ADEL_API_URL` + `ADEL_API_KEY` secret). It does **not** run the RAG brain itself. The pure, dependency-free cores of the billing/entitlement logic live in `*-core.js` (`entitlements-core.js`, `revenuecat-core.js`, `stripe-core.js`) so the `npm run test:unit` suite can exercise the invariants without the Admin SDK. **Region is a single source of truth in `functions/region.js`**: currently `me-central1` (Doha) as an interim, with `me-central2` (Dammam, in-Kingdom) as the PDPL target — migrating means flipping that literal plus the firebase.json rewrites and `billing.js` (see `office/RUNBOOK-pdpl-me-central2.md`). exports `chat` and several billing/account functions. The `chat` function is a thin gateway: it owns auth (verifies the Firebase ID token), entitlement checks, the free-tier daily quota (`rag/dailyquota.js`), the abuse rate-limiter (`rag/ratelimit.js`) and input guards (`rag/guards.js`), then **proxies the turn server-to-server** to the standalone Captain Adel service (`ADEL_API_URL` + `ADEL_API_KEY` secret). It does **not** run the RAG brain itself. Other exports: `content.js` (`protectedContent` — gated PDF/library delivery), `stripe.js` (web billing), `revenuecatWebhook.js` (iOS IAP → entitlement + `linkRevenueCatIdentity`), `reminders.js` (medical/flight-review push), `school.js` (B2B licences), `staff.js` (a tiny server-side owner/staff allowlist that resolves to full access without a billing grant — see security note below). `*-core.js` siblings (`entitlements-core.js`, `stripe-core.js`, `revenuecat-core.js`) hold the pure logic that `tests/unit/` exercises without the Firebase runtime. The deployable region is the single source of truth in `functions/region.js`: target is **me-central2** (Dammam, in-Kingdom), with an interim fallback to **me-central1** until Google grants me-central2 access — `firebase.json`'s function rewrites must match.

3. **Captain Adel, "the brain"** (standalone Node/Express RAG service, `captadel.com`). This is the single source of truth for the assistant. It now lives in its **own repository**, `FlyGACA/Captain-Adel` — it used to be a `captadel/` git subtree here. Key layout under its `src/brain/`: `answer.js` (orchestrator), `retrieve.js` + `bm25.js` + `embeddings.js` (BM25 retrieve-then-read), `grounding.js` (citation/grounding enforcement), `guards.js` + `ratelimit.js` (the service-side copies that mirror the gateway's), `route.js` (language/provider routing), `providers/` (`gemini` agentic function-calling for English; a set of in-Kingdom Arabic providers — `allam`, `jais`, `fanar`, `qwen`, `commandr`, all on the `openai-compatible` retrieve-then-read base), `system-prompt.js` + `tenants.js` (product-neutral core prompt with per-product framing for `captadel` vs `flygaca`), `_chunks.json.gz` (the GACAR corpus). Routing (`route.js`): an explicit `provider` is honoured; `auto`/unset sends Arabic-dominant questions (≥40% Arabic letters) to the first configured in-Kingdom Arabic provider (`ARABIC_PROVIDERS`, ALLaM first) and everything else to Gemini. **RAG is the source of truth for facts** — the model answers only from retrieved passages. Every change is **eval-gated** (`captadel/evals/`). `answer.js` (orchestrator), `retrieve.js` + `bm25.js` (BM25 retrieve-then-read), `route.js`

(language/provider routing), `grounding.js` / `guards.js` / `ratelimit.js`, `providers/` (English → `gemini` agentic function-calling; Arabic/in-Kingdom → `allam` retrieve-then-read; plus `fanar`, `jais`, `qwen`, `commandr`, `openai-compatible` fallbacks), `system-prompt.js` + `tenants.js` (product-neutral core prompt with per-product framing for `captadel` vs `flygaca`), `_chunks.json.gz` (the GACAR corpus). **RAG is the source of truth for facts** — the model answers only from retrieved passages. Every change is **eval-gated** (`captadel/evals/`).

The `/v1/chat` contract is `{ message, history, product, provider, session }` → `{ answer, sources: [{ citation, url }] }`. The gateway and the browser (`assets/js/chat.js`) both speak this shape.

Data flow & content

- **`assets/data/`** holds the library as JSON (GACAR index, aerodromes, charts, ebooks, ground school, definitions, search index, plus `parts/` — the 74 GACAR documents — and `library/` foreign references). Pages fetch these at runtime — hence the must-serve-over-HTTP rule. `scripts/check-data.js` enforces the index counts and structure the README advertises; if you change library content, run `npm run check:data` and update both. The corpus is machine-extracted from GACA PDFs, so `check:content` triages OCR/extraction defects and `update:sources` keeps the manifest (`assets/data/sources.json`) in sync with GACA's publications; `build:defs` derives `definitions-index.json` from `parts/part-1.html`.
- **`assets/js/`** is one module per page/feature — page modules (`library.js`, `chat.js`, `logbook.js`, `dashboard.js`, `settings.js`…), shared services (`entitlements.js`, `billing.js`, `firebase-config.js`, `auth.js`, `native-bridge.js`), and the large `tools-*.js` family backing the ~36 flight calculators/utilities under `tools/` (e.g. `tools-e6b.js`, `tools-crosswind.js`, `currency.js` for GACAR Part 61 currency). No build — files are loaded directly by the pages.
- **`assistant/`** is the human-authored source of truth for Captain Adel's persona, system prompt and knowledge-base scope (`captain_adel_system_prompt.md`, `knowledge_base_scope.md`, `CHARACTER_SHEET.md`). The deployed prompt lives in `captadel/src/brain/`; keep them in sync.

Backend data model & security

Firestore (Dammam region) uses **strict per-user isolation** — a pilot can only read/write the tree under their own `users/{uid}` (profile + `logbook/` subcollection). The crucial invariant in `firestore.rules`: the **`entitlement` field is server-only** — clients may never add, remove or alter it, so nobody can self-grant a paid plan; entitlements are written only by Cloud Functions via the Admin SDK. `waitlist/` is create-only and never client-readable. Server-only collections (e.g. `adelQuota`) are denied to all clients. `tests/rules.test.js` covers these guarantees.

The one sanctioned way to grant access without a billing record is `functions/staff.js`'s owner/staff allowlist — and it preserves the same invariant: it is evaluated **only against a decoded, `email_verified` Firebase ID token** (never client input), so a client cannot add itself. It has a client-side UX counterpart in `assets/js/store.js` (`TESTER_EMAILS`); keep the two lists in sync.

PDPL: real user questions are personal data and must be processed in-Kingdom — Captain Adel deploys to a KSA region. Keep this in mind for anything touching user data or model calls.

Conventions

- **Shared chrome is generated — never hand-edit it.** The `<header class="site-nav">` and `<footer class="site-footer">` blocks in every page are stamped from `partials/header.html` and `partials/footer.html` by `scripts/build-chrome.js`. Edit the partials, then run `npm run build:chrome`. CI (`check:chrome`) fails the PR if pages drift. The stamper fixes relative path depth and the active-nav marker per page, and skips stub/redirect pages that carry no chrome. It only touches pages in `.`, `tools/`, `guides/`, `study/`, `packs/`.
- **Bilingual + RTL** is a first-class requirement (English / العربية). The site translates on the fly from `data-en` / `data-ar` attributes (engine in `assets/js/landing.js`), so new user-facing copy needs **both** — `npm run check:i18n` fails the PR on any half-translated string. The assistant's Arabic path auto-routes to an in-Kingdom Arabic provider (ALLaM by default).
- **Service worker** (`sw.js`) is freshness-aware and served `no-cache`; bump its version when changing cached assets so the PWA refreshes.
- **Security headers / CSP** are defined in `firebase.json` (strict CSP, `frame-ancestors none`, HSTS). New external origins (scripts, connect, frames) must be added there explicitly.

CI

`.github/workflows/ci.yml` runs on every PR:

- `chrome` — chrome-in-sync guard (`build-chrome.js --check`)
- `data` — library integrity (`check-data.js`)
- `links` — fast checks: `test:unit` + `check-i18n.js` + `check-links.js`
- `e2e` — Playwright smoke-loads every page + critical flows; posts failing tests as a PR comment
- `rules` — Firestore rules against the emulator (Java 21+); posts the log tail on failure

(Captain Adel's own eval/brain CI now runs in its `FlyGACA/Captain-Adel` repo, not here.)

The `deploy` job (Hosting + rules) runs on push to `main` only once a `FIREBASE_TOKEN` secret is set; Functions are deployed manually from `office/RUNBOOK-*.md` because they need the Gemini/Adel secrets. A second workflow, `.github/workflows/update-sources.yml`, keeps the GACA corpus in sync via `update-sources.js`. `.github/workflows/ci.yml` runs on every PR: `chrome` (chrome-in-sync guard), `data` (`check-data.js`), `links` (fast guards bundled — `test:unit` node tests, then `check:i18n` and `check:links`), `e2e` (Playwright smoke-loads every page + critical flows; posts failing tests as a PR comment), and `rules` (Firestore emulator; posts a log tail on failure). The `deploy` job (Hosting + rules, project `flygaca-app`) runs on push to `main` only once a `FIREBASE_TOKEN` secret is set; Functions are deployed manually from `office/RUNBOOK-*.md` because they need the Gemini/Adel secrets. A separate `update-sources.yml` workflow periodically drift-checks the GACA source manifest.

Reference

- `ROADMAP.md` — full product/phasing plan; `PHASE0.md` — origin/phase notes.
- `office/` — runbooks and specs. Deploy/ops: `RUNBOOK-deploy.md`, `RUNBOOK-captain-adel.md`, `RUNBOOK-arabic-provider.md`, `RUNBOOK-ios.md`, `RUNBOOK-launch.md`, `RUNBOOK-source-updates.md`, `RUNBOOK-security-rollout.md`, `RUNBOOK-vps-hardening.md`. PDPL/region: `RUNBOOK-pdpl-me-central2.md`. Setup: `SETUP-firebase.md`, `SETUP-vps.md`, `SETUP-entity.md`. Extraction: `RUNBOOK-captadel-extraction.md` (how `captadel/` was split out into `FlyGACA/Captain-Adel`). Consult these before deploy/setup work.
- `docs/` — customer-success playbooks (onboarding, renewal, at-risk, expansion, QBR, health scoring); see `CUSTOMER_SUCCESS.md` for the index.
- `functions/README.md`, `tests/README.md`, `assets/README.md` — per-area detail.
- `office/` — runbooks and specs: `RUNBOOK-deploy.md`, `RUNBOOK-captain-adel.md`, `RUNBOOK-ios.md`, `RUNBOOK-launch.md`, `SETUP-firebase.md`, `RUNBOOK-captadel-extraction.md` (how `captadel/` was split out into `FlyGACA/Captain-Adel`). Consult these before deploy/setup work.
- `docs/` — customer-success playbooks (onboarding, renewal, expansion, at-risk, QBR, health-scoring); see also `CUSTOMER_SUCCESS.md` and `CONTENT-QA.md` at the root.
- `functions/README.md`, `tests/README.md` — per-area detail.